```
"""
================================================================================
JAGAPRIYAN B
CH.SC.U4CSE24216
LAB EXERCISE 5: LINEAR REGRESSION

Topics Covered:
  Exercise 1: Chicago Taxi Trips Fare Prediction (Linear & Multiple Linear Regression)
  Exercise 2: Body Weight Prediction using Height, Age, and Gender
================================================================================
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

print("="*70)
print("EXERCISE 5.1: CHICAGO TAXI TRIPS FARE PREDICTION")
print("="*70)

# 1. Dataset: Representative Chicago Taxi Trips dataset with key features:
# Trip Miles, Trip Seconds, Pickup/Dropoff Community Areas, Fare
np.random.seed(42)
n_samples = 500

trip_miles = np.random.uniform(0.5, 25.0, n_samples)
trip_seconds = trip_miles * np.random.uniform(100, 180, n_samples) + np.random.normal(60, 20, n_samples)
# Base fare calculation similar to Chicago Taxi fare structure: $3.25 base + $2.25/mile + time charge + noise
taxi_fare = 3.25 + (2.25 * trip_miles) + (0.005 * trip_seconds) + np.random.normal(0, 1.5, n_samples)
taxi_fare = np.maximum(taxi_fare, 3.25) # Minimum fare

taxi_data = pd.DataFrame({
    'trip_miles': np.round(trip_miles, 2),
    'trip_seconds': np.round(trip_seconds, 0),
    'fare': np.round(taxi_fare, 2)
})

print("First 5 rows of Chicago Taxi Trips dataset:")
print(taxi_data.head())
print("\nDataset Info:")
print(f"Shape: {taxi_data.shape}")
print(f"Missing values:\n{taxi_data.isnull().sum()}")

# Independent and Dependent variables
X_taxi = taxi_data[['trip_miles', 'trip_seconds']]
y_taxi = taxi_data['fare']

# Split dataset
X_train_taxi, X_test_taxi, y_train_taxi, y_test_taxi = train_test_split(
    X_taxi, y_taxi, test_size=0.2, random_state=42
)

# Fit Multiple Linear Regression Model
taxi_model = LinearRegression()
taxi_model.fit(X_train_taxi, y_train_taxi)
y_pred_taxi = taxi_model.predict(X_test_taxi)

# Model Evaluation
mse_taxi = mean_squared_error(y_test_taxi, y_pred_taxi)
rmse_taxi = np.sqrt(mse_taxi)
r2_taxi = r2_score(y_test_taxi, y_pred_taxi)

print("\n--- Model Coefficients & Evaluation ---")
print(f"Intercept (b0): {taxi_model.intercept_:.4f}")
print(f"Coefficients (b1: trip_miles, b2: trip_seconds): {taxi_model.coef_}")
print(f"Mean Squared Error (MSE): {mse_taxi:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse_taxi:.4f}")
print(f"R-squared Score (R2): {r2_taxi:.4f}")

# Visualization: Actual vs Predicted Fare
plt.figure(figsize=(8, 5))
plt.scatter(X_test_taxi['trip_miles'], y_test_taxi, color='blue', alpha=0.6, label='Actual Fare')
plt.scatter(X_test_taxi['trip_miles'], y_pred_taxi, color='red', marker='x', alpha=0.7, label='Predicted Fare')
plt.title("Chicago Taxi Trips: Trip Miles vs Fare")
plt.xlabel("Trip Miles")
plt.ylabel("Fare ($)")
plt.legend()
plt.grid(True)
```

```
plt.grid(True)
plt.show()


print("\n" + "="*70)
print("EXERCISE 5.2: BODY WEIGHT PREDICTION (HEIGHT, AGE, GENDER)")
print("="*70)

# Exact Table from the Lab Activity Book (Page 20)
# Weight | Height | Age | Gender
data_dict = {
    'Weight': [79, 69, 73, 95, 82, 55, 69, 71, 64, 69],
    'Height': [1.80, 1.68, 1.82, 1.70, 1.87, 1.55, 1.50, 1.78, 1.67, 1.64],
    'Age': [35, 39, 25, 60, 27, 18, 89, 42, 16, 52],
    'Gender': ['Male', 'Male', 'Male', 'Male', 'Male', 'Female', 'Female', 'Female', 'Female', 'Female']
}

df_person = pd.DataFrame(data_dict)
print("Dataset provided in Lab Manual:")
print(df_person)

# Encode Categorical Feature 'Gender': Male=1, Female=0
df_person['Gender_Encoded'] = df_person['Gender'].map({'Male': 1, 'Female': 0})

# Independent variables: Height, Age, Gender
X_person = df_person[['Height', 'Age', 'Gender_Encoded']]
# Dependent variable: Weight
y_person = df_person['Weight']

# Fit Linear Regression Model
weight_model = LinearRegression()
weight_model.fit(X_person, y_person)

# Predictions on the dataset
y_pred_person = weight_model.predict(X_person)

print("\n--- Regression Results ---")
print(f"Intercept (b0): {weight_model.intercept_:.4f}")
print("Coefficients:")
print(f"  Height (b1): {weight_model.coef_[0]:.4f}")
print(f"  Age (b2):    {weight_model.coef_[1]:.4f}")
print(f"  Gender (b3): {weight_model.coef_[2]:.4f} (1=Male, 0=Female)")

mse_p = mean_squared_error(y_person, y_pred_person)
r2_p = r2_score(y_person, y_pred_person)
print(f"\nMean Squared Error (MSE): {mse_p:.4f}")
print(f"R-squared Score (R2): {r2_p:.4f}")

# Display Comparison
df_results = pd.DataFrame({
    'Actual_Weight': y_person,
    'Predicted_Weight': np.round(y_pred_person, 2),
    'Residual (Error)': np.round(y_person - y_pred_person, 2)
})
print("\nActual vs Predicted Comparison:")
print(df_results)

# Test Prediction on a new sample: Height=1.75m, Age=30, Gender=Male (1)
sample_person = pd.DataFrame([[1.75, 30, 1]], columns=['Height', 'Age', 'Gender_Encoded'])
predicted_sample_weight = weight_model.predict(sample_person)
print(f"\nPrediction for new person (Height=1.75m, Age=30, Male): {predicted_sample_weight[0]:.2f} kg")

# Plotting: Actual vs Predicted Weight
plt.figure(figsize=(8, 5))
plt.plot(range(len(y_person)), y_person, 'bo-', label='Actual Weight')
plt.plot(range(len(y_pred_person)), y_pred_person, 'ro--', label='Predicted Weight')
plt.title("Actual vs Predicted Body Weight")
plt.xlabel("Sample Index")
plt.ylabel("Weight (kg)")
plt.legend()
plt.grid(True)
plt.show()
```